

SIH26122 — Detailed Build Plan

Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management

Problem statement owner: Oil India Limited

One-liner: An AI assistant that listens to what workers say happened on site and automatically keeps the official project plan accurate and up to date — instead of a planner doing it by hand, days late.

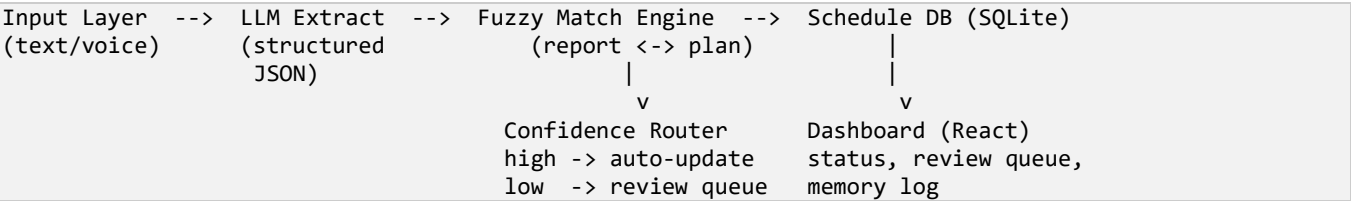
1. Restating the Problem Precisely

Three things are broken today:

1. Capture is informal — supervisors log progress in texts, diaries, Excel — whatever's easiest.
2. Translation is manual and slow — someone has to read that informal note and manually update the "official" schedule (MS Project / Primavera / Excel plan), days later.
3. No institutional memory — nobody systematically tracks how long tasks actually took vs. planned, so every new project re-estimates from scratch.

Your system needs to solve all three, in this order of priority: (1) capture → (2) match → (3) update → (4) memory. Judges will care most about (2), the matching, because that's the actual AI/hard part. Everything else is plumbing around it.

2. System Architecture (End to End)



Core components

1. Input Layer — chat-style text box (typed) + optional mic button (Whisper STT).
2. Extraction Layer — LLM call that turns free text into structured JSON: {task, quantity, location, date, raw_text}.
3. Matching Engine — fuzzy string match (rapidfuzz) between extracted task description and the schedule's official task names, producing ranked candidate matches with confidence scores.
4. Decision Router — if top match confidence is above threshold (e.g. 80%), auto-mark task complete with timestamp. If below threshold, push to a human review queue instead of guessing.
5. Schedule Database — SQLite table of the project plan (task ID, name, planned dates, status, actual completion date).
6. Memory/Analytics Layer — logs actual duration vs. planned duration per task type, so it can later say "pipe erection tasks average 2.3 days late historically."
7. Dashboard (React + Tailwind) — shows live schedule status, the review queue, and a simple analytics view.

3. Data Model (SQLite Schema)

schedule_items

Column	Type	Notes
id	INTEGER PK	
task_name	TEXT	official name, e.g. "Erect Line 24-XX"
location	TEXT	site/zone
planned_start	DATE	
planned_end	DATE	

Column	Type	Notes
status	TEXT	pending / in progress / done / flagged
actual_completion_date	DATE	nullable
linked_report_id	INTEGER	FK, nullable

reports

Column	Type	Notes
id	INTEGER PK	
raw_text	TEXT	what the supervisor typed/said
extracted_task	TEXT	LLM output
extracted_quantity	TEXT	
extracted_location	TEXT	
extracted_date	DATE	
matched_schedule_id	INTEGER	FK, nullable
confidence_score	FLOAT	0–1
review_status	TEXT	auto applied / needs review / rejected
created_at	TIMESTAMP	

task_history (the "memory" layer)

Column	Type	Notes
id	INTEGER PK	
task_type	TEXT	e.g. "pipe erection"
planned_duration_days	INTEGER	
actual_duration_days	INTEGER	
delay_reason	TEXT	optional, free text or tag

4. The Matching Logic in Detail (Your Differentiator — Spend Real Time Here)

Step 1 — LLM structured extraction

Prompt the LLM with a strict system instruction to return only JSON:

```
Extract the following fields from this site report. Return ONLY valid JSON, no prose.
Fields: task (string), quantity (string or null), location (string or null), date (ISO string or null)
Report: "finished the Line 24 pipe work today"
```

Step 2 — Candidate generation

Use `rapidfuzz.process.extract()` to compare `extracted_task` against every `task_name` in `schedule_items`, filtered first by location if available (narrows the search space and improves accuracy). Return top 3 candidates with token-sort-ratio scores.

Step 3 — Score fusion

Don't rely on string similarity alone — combine three signals into one confidence score:

- String similarity (`rapidfuzz.token_sort_ratio`) — 50% weight
- Location match (exact/partial match on site/zone) — 25% weight
- Date plausibility (report date falls within or near planned window) — 25% weight

$confidence = 0.5 \times string_score + 0.25 \times location_score + 0.25 \times date_score$

Step 4 — Decision routing

- confidence ≥ 0.80 → auto-mark task done, set `actual_completion_date`, log to `task_history`.
- $0.5 \leq \text{confidence} < 0.80$ → push to review queue with the top 3 candidates shown side by side, ranked.
- confidence < 0.5 → flag as "no clear match," visible in a separate queue, don't guess at all.

This threshold logic is the single most "judge-convincing" part of the build — show it explicitly in the demo with a slider or visible score.

5. Tech Stack (Locked In)

Layer	Tool	Why
Backend	FastAPI (Python)	fast to build, async-friendly, good with LLM SDKs
Database	SQLite	zero-setup, fine for a demo dataset of 15–20 tasks
LLM	Claude or GPT API	structured extraction
Fuzzy matching	rapidfuzz	fast, well-documented, pure Python
STT (stretch)	Whisper API	voice input for supervisors
Frontend	React + Vite + Tailwind	matches your existing Sahaara stack — reuse component patterns
Charts (memory view)	Recharts	simple bar chart of planned vs actual

6. 36-Hour Build Plan, Hour by Hour

Hours 0–4: Setup + synthetic data

- Set up repo, FastAPI skeleton, SQLite schema, seed script.
- Build a synthetic project schedule: 15–20 line items across 3–4 zones (e.g. "Erect Line 24-XX," "Trench excavation Zone B," "Valve installation Site C"), with realistic planned dates.
- Write 10–15 sample "supervisor reports" in varied phrasing to test matching against later (this is your test set — write it now, not later).

Hours 4–10: Extraction pipeline

- Wire the LLM API call for structured extraction.
- Test against your 10–15 sample reports; log failures (LLM misreads task, wrong date parsing, etc.) and refine the prompt.
- Handle edge cases: missing date (default to "today"), ambiguous location.

Hours 10–16: Matching engine

- Implement rapidfuzz candidate generation.
- Implement the score-fusion formula (string + location + date).
- Implement the threshold router (auto-update / review / no-match).
- Test end-to-end: report in → structured JSON → matched schedule item → confidence score, on all sample reports.

Hours 16–20: Database wiring + auto-update

- Connect matched, high-confidence reports to actually update `schedule_items.status` and `actual_completion_date`.
- Log every completed task into `task_history` with planned vs actual duration.

Hours 20–28: Dashboard

- Schedule table view: task, status (color-coded), planned vs actual date.

- "Needs review" queue: shows raw report text + top 3 candidate matches + confidence, with accept/reject buttons for a human.
- Simple memory/analytics view: bar chart of average delay by task type (this is your "over time it learns" payoff — even with synthetic history it tells a strong story).

Hours 28–32: Voice input (stretch, only if ahead of schedule)

- Add a mic button, wire Whisper API, feed transcribed text into the same extraction pipeline. No separate logic needed — voice is just another text source.

Hours 32–36: Polish + demo script

- Write and rehearse a live demo (see Section 8).
- Add loading states, error handling, and clean up the UI.
- Prepare a 1-slide architecture diagram and a 1-slide "why this matters" (real project delays, cost overruns from stale plans).

7. Team Role Split (6 Members)

Role	Responsibility	Best fit if...
Backend/API lead	FastAPI endpoints, LLM integration, extraction prompt tuning	comfortable with Python + API calls
Matching engine owner	rapidfuzz logic, score fusion, threshold tuning	enjoys logic/algorithm work, will own the demo's "wow" moment
Database/data lead	Schema, seed data, synthetic schedule + sample reports	detail-oriented, good with structuring realistic test data
Frontend lead	React dashboard: schedule table, status colors	has React/Tailwind experience
Frontend/UX support	Review queue UI, analytics/memory chart	second frontend person, Recharts
Integration + demo lead	Wires everything together, owns Whisper stretch goal, writes/rehearses demo script, builds slides	strong communicator, comfortable presenting

Since your team's current strength is "vibe coding + APIs," lean into that for the extraction and matching layers — that's exactly where LLM APIs + a fuzzy-matching library get you 80% of the way with modest code.

8. Demo Script (Rehearse This Exactly)

1. Open on the problem (30 sec): "Right now, site progress lives in texts and Excel sheets. The official plan is always out of date." Show a messy example.
2. Live input #1 — clean match: Type "finished the Line 24 pipe work today." Show it auto-extracting fields, matching with 92% confidence, and the schedule table updating live with a green checkmark.
3. Live input #2 — ambiguous match: Type something vaguer, e.g. "did the valve thing at site C." Show it landing in the review queue with 2–3 ranked candidates instead of guessing wrong — this is the trust-building moment, emphasize it explicitly.
4. Live input #3 — voice (if built): Speak a report into the mic, show the same pipeline firing.
5. Memory view: Switch to the analytics tab, show the delay-by-task-type chart: "Over time, this tells future projects that pipe erection tasks typically run 2 days over — so the next plan can budget for it."
6. Close on impact (30 sec): No more days-late updates, human-in-the-loop for anything uncertain, and a growing dataset for better future planning.

9. Risks and How to Pre-empt Them

Risk	Mitigation
LLM extraction is inconsistent on messy input	Constrain the prompt tightly, give 2–3 few-shot examples in the system prompt
Fuzzy match gives false positives on similar-sounding tasks	Weight location + date alongside string similarity, not string alone
Judges ask "what if the LLM API is down/offline"	Have a rule-based fallback (keyword match) ready to mention, even if not fully built
Demo network fails live	Pre-record a backup video of the exact same demo flow
Judges probe real-world data privacy/security	Have a one-line answer ready: on-prem/local LLM deployment is possible for sensitive project data

10. What to Build First if Time Runs Short

Priority order if you're behind schedule:

1. Extraction + matching + auto-update (the core loop) — non-negotiable.
2. Review queue for low-confidence matches — this is what makes it trustworthy, not just clever.
3. Dashboard polish.
4. Memory/analytics chart.
5. Voice input — cut this first if squeezed, it's a nice-to-have, not the core ask.